Requirement 1

Design 1

Create class/interface:

1. OnHitEffect
2. EffectProvider
3. EffectSlot
4. WeaponWithEffects
5. InstallEffectAction
6. PoisonOnHitEffect
7. FrostbiteOnHitEffect
8. YewberryItem
9. SnowGround

Pro	Cons
Supports future scalability — can easily extend to arrow effects, traps, or skills beyond weapon	Increased number of interfaces and abstract components may make debugging harder at first.
	Must manage the lifecycle of each effect (e.g., duration, expiration) carefully to avoid unexpected stacking.
	Harder to extend → adding a new teleport type requires modifying TeleportAction.

Design 2

Create class

1. CoatWeaponAction
2. Poisoning
3. Frosting
4. CountinuousWarmthloss
5. Snowcoating
6. Yewberrycoating
7. Coatable interface
8. Coating interface

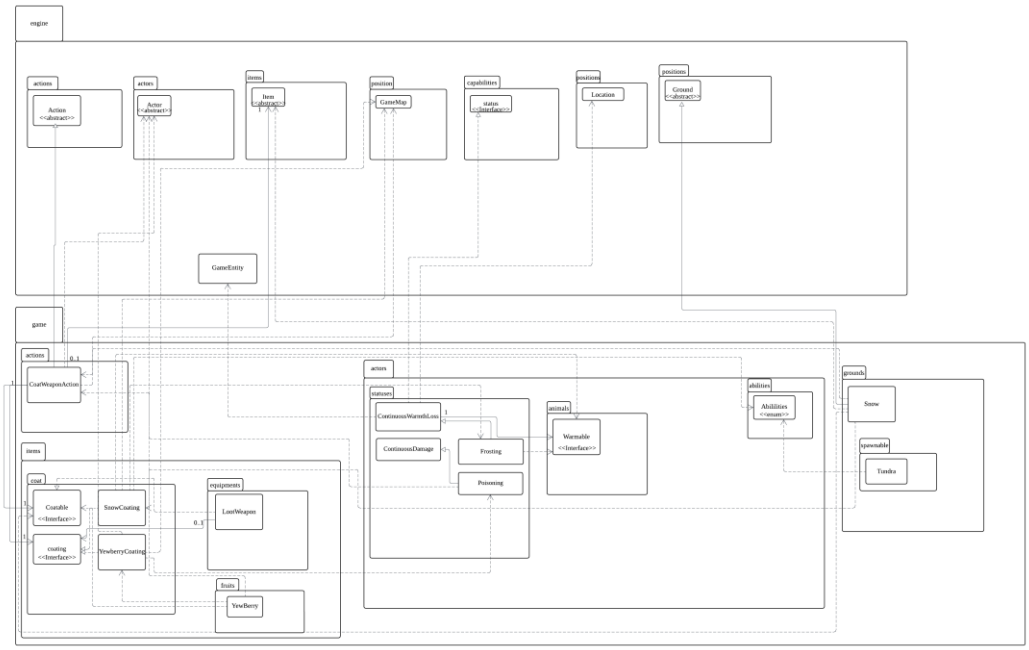
Pro	Cons
Each coating type (YewberryCoating, SnowCoating) directly maps to its source	Limited reusability — the design focuses mainly on weapon coatings; reusing

item or environment, making debugging and logic tracing straightforward.	effects for traps or skills would require additional integration.
Coatable and Coating interfaces eliminate the need for instanceof and allow flexible interaction with any coatable weapon.	Needs careful handling when the actor moves across maps or drops weapons to avoid null pointer issues or effect persistence errors.
coating effects are separated into layers — Coating handles weapon coating, while Poisoning and Frosting handle turn-based effects through ContinuousDamage and ContinuousWarmthLoss.	Must follow the contract, (e.g., Precondition, Postcondition etc...)

Choose:

Design 2 was chosen because it provides a cleaner and more maintainable structure through abstraction and clear separation of concerns. By introducing the Coating and Coatable interfaces, the design allows any weapon to be coated without relying on instanceof or complex conditional checks, fully supporting polymorphism. Each coating type (e.g., YewberryCoating, SnowCoating) encapsulates its own behavior while delegating continuous effects to independent status classes (Poisoning, Frostbite), which improves modularity and testing.

This design follows the Open/Closed Principle by enabling new coatings or effects to be added without modifying existing classes. It also reduces code duplication and tightly integrates with the existing attack system. Although the design is slightly more abstract and requires more classes, its long-term benefits in scalability, clarity, and adherence to SOLID principles make it the more sustainable choice for REQ4.



1. What classes will exist in the extended system
    a. Interface:
        i. Coatable
        ii. Coating
        iii. Warmable
    b. Abstract class:
        i. ContinuousWarmthloss
        ii. continuousDamage
    c. Enum class:
        i. Abilities
    d. Concrete Class:
        i. CoatWeaponAction
        ii. SnowCoating
        iii. YewberryCoating
        iv. LootWeapon
        v. YewBerry
        vi. Frosting
        vii. Poisoning
        viii. Snow
        ix. Tundra
2. Roles & Responsibilities
    a. Coatable → Implemented by weapons that can be coated. Provides the ability to apply, replace, and remove coatings dynamically during gameplay. Ensures each weapon can hold at most one active coating at a time.
    b. Defines the generic behaviour of coatings that can be applied to Coatable weapons. Each implementation determines its own on-hit effect (e.g., poison or frostbite) and lifespan.
    c. Warmable → Implemented by actors that can gain or lose warmth. Provides an interface for effects such as Frosting to decrease the actor's warmth over time.
    d. ContinuousWarmthLoss → Abstract class that handles continuous warmth reduction effects. Manages turn-based ticking, duration countdown, and removal when the effect expires.
    e. ContinuousDamage → Abstract class for continuous HP loss effects. Provides reusable logic for periodic damage and expiration, allowing derived statuses such as Poisoning to apply consistent turn-based damage.
    f. Abilities → Stores unique capability flags for actors and grounds (e.g., COLD_RESISTANT). Used to determine immunity or resistance when applying coating effects like Frosting.
    g. CoatWeaponAction → Action class enabling an actor to apply a coating onto a Coatable weapon. Handles coating replacement logic, consumes

the coating source (if it is an item like YewBerry), and ensures proper interaction between item-based and environment-based coatings.

h. SnowCoating → Concrete implementation of the Coating interface that applies frostbite effects.
i. YewberryCoating → Concrete implementation of the Coating interface that applies poison effects.
j. LootWeapon → Represents weapons that can be coated. Integrates with the coating system by invoking the coating's effect after performing its normal attack logic.
k. YewBerry → Consumable item that provides a poison coating. When used with CoatWeaponAction, it is consumed and applies YewberryCoating (poison effect) to the selected weapon.
l. Frosting → Concrete status extending ContinuousWarmthLoss. Applies a per-turn reduction of warmth to the affected actor for a fixed duration, simulating cold damage over time.
m. Poisoning → Concrete status extending ContinuousDamage. Inflicts health loss on the affected actor each turn for a set number of turns before automatically expiring.
n. Snow → Ground type that provides coating opportunities when the actor stands on it. Exposes CoatWeaponAction to the player, enabling coating weapons with SnowCoating directly from the environment.
o. Tundra → A ground subclass that represents a frozen environment. May generate Snow tiles and thematically supports the coating system by acting as a natural habitat for cold-related effects.

3. How these classes relate to and interact with the existing system.
    a. CoatWeaponAction integrates with the existing action menu. When an actor holds a Yewberry-based coating source or stands on a Snow tile, the action appears and applies a coating to a Coatable weapon. Applying a yewberry coating consumes the item from the inventory; applying a snow coating uses the ground as the source. Torches are excluded (isCoatable() == false). Re-coating replaces the current coating.
    b. Weapon attack flow LootWeapon implements Coatable. During attack(), the weapon deals its normal damage first, then (if coated) invokes the Coating's on-hit logic.
        i. YewberryCoating → Poisoning: on hit, inflicts 4 HP per turn for 5 turns on the target. The poison stacks across multiple hits.
        ii. SnowCoating → Frosting: on hit, applies a frostbite stack that lasts 3 turns; each active stack deducts 1 warmth per turn.
    c. Poisoning (extends ContinuousDamage) and Frosting (extends ContinuousWarmthLoss) are registered in the engine's status system and tick automatically each round. Effects stack additively: multiple

poisonings sum their per-turn HP loss; multiple frostings sum their +1 warmth-loss per stack.

d. Actors implementing Warmable expose warmth getters/setters used by Frosting. Creatures spawned from Tundra are created with the capability Abilities.COLD_RESISTANT, which makes them immune to Frosting's warmth loss. Other actors without this capability are affected normally and include their permanent coldness (spawn attribute) in the per-turn deduction.

4. How the classes interact to deliver functionality.
    a. When an actor (Player or NPC) is (i) standing on a **Snow** tile or (ii) carrying a **Yewberry**-based source, the source surfaces **CoatWeaponAction** via its allowableActions(...). **Torches** report isCoatable() == false, so the action does not appear for them.
    b. The actor selects CoatWeaponAction on a Coatable weapon. The action calls applyCoating(Coating) on the weapon. If the weapon already has a coating, the new coating replaces the old one. If the source is a Yewberry item, it is consumed from inventory; Snow uses the ground as the source (no inventory item to remove).
    c. During LootWeapon.attack() the weapon performs base hit/damage first, then—if a coating is installed—invokes the coating's on-hit behaviour:
        i. **YewberryCoating → Poisoning**: apply a status that deals **4 HP per turn for 5 turns**. **Stacks** additively across hits.
        ii. **SnowCoating → Frosting**: apply a status that lasts **3 turns**, each stack deducting **1 warmth per turn**. The per-turn deduction is **(permanent coldness + active frost stacks)**, matching the example progression in the spec.
    d. Actors implementing **Warmable** allow Frosting to modify warmth. Creatures **spawned on Tundra** start with **Abilities.COLD_RESISTANT**, making them **immune to Frosting's warmth loss**; other actors are affected normally (including their permanent coldness).
    e. Player stands on **Snow** or uses a **Yewberry** → chooses **CoatWeaponAction** → weapon gets coated (old coating, if any, is replaced) → player attacks → coating applies Poisoning or Frosting on hit → statuses tick each turn, stacking and expiring according to their durations.